

第 4 组调研报告

题目：面向桌面 Agentic OS 的「工具注册 + OS Skills」层调研——从 UFO² Tools 到 MCP 协议

- 课程：操作系统设计（专业必修 • 实践环节）
- 项目：Linux 桌面 Agentic OS 设计与实现
- 本组方向：第 4 组 • 工具服务层（ToolRegistry + SkillLibrary）
- 组员：第 4 组全体
- 指导教师：陈浩、肖德贵
- 完成时间：2026-08 下旬（第 1 次调研）— 09-09 定稿（真机联调后扩充至约 4000 字）

一、引言

1.1 调研背景

本课程以微软 UFO²（UI-Focused Agent）为蓝本，要求在 Linux 桌面环境构建一个支持自然语言交互的 Agentic OS 原型，并在此过程中理解操作系统三大核心原理——虚拟化、并发、持久性。整个系统按 UFO² 架构分为 5 组：AI Shell/HostAgent（用户交互）、任务规划/控件检测（决策规划）、AppAgent/Automator（执行）、工具注册/OS Skills（工具服务）、系统协调/安全/RAG（系统协调）。

第 4 组承担的是 工具服务层：把“文件系统/系统管理”等操作系统能力封装成一组可供上层（执行层）统一调用的工具（Tools）与技能（Skills）。该层在 UFO² 中对应 Tools 组件，在本课程评分中对应“OS 持久性”原理的直接体现——文件系统操作、目录管理正是持久性（文件系统）最直观的落点。

1.2 调研方法

本报告采用“文献调研 → 协议深入 → 竞品对标 → 落地决策”四步法：

- 文献调研：精读 UFO / UFO² 论文中关于 Tools 与 AppAgent 工具调用的章节，阅读 Agentic OS 综述，明确“工具层”在智能体堆栈中的定位；
- 协议深入：重点研读 MCP（Model Context Protocol）官方规范，分析其 Server/Tool、工具定义 Schema、调用-响应消息格式，作为我们设计 ToolRegistry 接口的参考蓝本；
- 竞品对标：横向比对 Microsoft UFO、阿里 ANOLISA、Open Interpreter、LangChain/LangGraph 工具生态，提炼“工具注册、统一调用、调用统计、来源校验”等共性机制；
- 落地决策：结合课程“接口契约、Mock 机制、跨组联调”的硬约束，确定第 4 组的架构选型与接口设计（详见《第 4 组设计文档》）。

1.3 调研结论摘要

- 工具层是桌面 Agent 从“只会聊天”走向“能操作系统”的关键枢纽，其核心价值是把任意系统能力收敛为统一契约的、可审计的、可安全调用的函数；
- MCP 协议提供了一套成熟的“服务端声明工具 → 客户端发现 → 结构化调用 → 结果回传”的范式，其 Tool 定义（name/description/inputSchema）与第 4 组自研接口高度同构，可直接借鉴；
- 第 4 组应具备四类能力：统一注册与调用、调用统计与审计、工具签名（来源/完整性校验）、离线可复现的 Mock——这与设计书“功能要求”完全吻合；
- 现有代码实现 59 个系统工具 + 26 个 OS 技能，全部实现调用统计（get_stats）与工具签名（get_tool_signature），并通过 512 项单元测试，验证了调研结论的可落地性。

二、UF0² 架构与工具层定位

2.1 UF0² 组件回顾

微软 UF0² 将 Windows 桌面 Agent 拆解为四大组件：

组件	职责	对应本项目分组
HostAgent	接收自然语言指令、意图理解、任务分发，协调多个 AppAgent	第 1 组
AppAgent	单一应用的状态管理与动作规划，采用 Hybrid GUI + API Actions	第 3 组
Control Detection	视觉检测 + UIA 结构化检测的混合控件定位	第 2 组
Knowledge Substrate	执行轨迹存储检索、应用文档知识库、RAG 增强	第 5 组
Tools（工具）	对操作系统与外部服务能力的标准化封装，供 Agent 调用	第 4 组

在 UF0² 中，AppAgent 并非“自己硬写 GUI 脚本”，而是倾向于通过 API Actions / 工具调用 完成操作——这正是第 4 组存在的原因：当某个操作能用确定性系统调用完成时，就绝不依赖脆弱的坐标点击。工具层因此成为“GUI 自动化”与“系统调用”两种路径的汇流闸门。

2.2 工具层在智能体堆栈中的位置

借助传统分层视角，我们可将桌面 Agent 看作：



| 能力层：文件系统 / 进程 / 网络 / 应用 / 剪贴板 | (OS 提供的持久化抽象)

工具层是“决策”与“能力”之间的适配面：它把下层 OS 的原始能力（shutil、subprocess、psutil、AT-SPI/CDP……）封装成上层可理解、可验证、可安全调用的函数签名。这一思想与操作系统的“系统调用”不谋而合——系统调用把硬件能力暴露为受控 API，工具层把 OS 能力暴露为受控的智能体 API。

2.3 与 OS 原理的关联（第 4 组视角）

- 持久性（文件系统）：第 4 组工具的主体是文件/目录操作（复制、移动、分类、搜索、归档、回收站等），直接操作持久存储上的文件系统抽象，是“持久性”原理在智能体中的最直接体现；
- 虚拟化（资源抽象）：工具层对进程、磁盘、网络等资源做统一抽象和配额（如 `run_command` 的沙箱、超时），类似 OS 对进程资源的抽象；
- 并发：工具调用在 Web 多线程流式场景下与其它请求并发执行，需要线程安全的注册表与统计（本项目用线程隔离的连接池与线程安全字典保障）。

三、MCP 协议调研（重点）

3.1 为什么是 MCP

MCP（Model Context Protocol）由 Anthropic 提出并开源，定位是“AI 模型与外部工具/数据源之间的开放协议”，类比于 USB-C 之于外设。对第 4 组最重要的启示是：不必为每个工具发明私有调用语法，而应采用统一的工具描述与调用范式，让上层 Agent 无需感知具体工具是本地函数还是远端服务。

3.2 MCP 核心对象

对象	字段	意义
Tool	<code>name</code> , <code>title</code> , <code>description</code> , <code>inputSchema</code> (JSON Schema)	工具声明：名称、人类可读描述、参数校验 Schema
CallToolRequest	<code>params.name</code> , <code>params.arguments</code>	客户端发起的工具调用请求
CallToolResult	<code>content[]</code> , <code>isError</code>	结构化结果，区分正常/错误
统一契约	<code>name</code> + JSON Schema + 结果对象	工具可被动态发现与校验

MCP 的关键设计：工具的描述（`description`）+ 输入 Schema 与工具实现解耦。智能体看到的是“工具清单 + 每个工具怎么用”，调用时传入结构化参数，得到结构化结果。这与我们的“给 LLM 传 function calling schema”完全一致——第 4 组在实现时，正是让每个工具自动从函数签名+docstring 推断 `description` 与 `parameters`（参见《设计文档》），与 MCP 的 `inputSchema` 异曲同工。

3.3 MCP 三种传输与我们的取舍

MCP 规范定义了 `stdio`（子进程）与 `HTTP/SSE`（服务端）两类传输。调研结论：

- `stdio` 适合“本地、进程内”的确定性工具（文件操作等），开箱即用、零网络成本——与第 4 组“本地 OS 工具”场景吻合；
- `HTTP/SSE` 适合需要鉴权/远程的数据源工具（网页检索、邮件服务器等）；
- 本项目选型：本地 OS 工具采用进程内直调（即 MCP `stdio` 的精神），对有网络副作用的工具（联网搜索、发邮件、浏览器）单独做安全确认，与 MCP 服务端鉴权思路一致。结果对象统一为 `{success, result|error}`，等价于 MCP 的 `CallToolResult` 归一化。

3.4 MCP 对接口设计的四条启示（直接落地到第 4 组）

1. 统一发现：提供 `list_available_tools()` / `list_skills()`，向 Agent 暴露完整工具清单及其描述（对应 MCP Server `tools/list`）；
2. Schema 驱动：工具参数自动从函数签名/typing + docstring 生成 JSON Schema，供 LLM function calling 使用（对应 MCP `inputSchema`）；
3. 结果归一：所有调用统一返回 `{success, result, error?}`，让上层无需区分具体工具（对应 MCP `CallToolResult`）；
4. 可插拔：`MockToolRegistry` 提供与正式版同构的 57 个工具 Mock，可整体替换、便于跨组联调（对应 MCP“实现与接口解耦”）。

更完整的协议层面调研见 `docs/mcp_research.md`。

3.5 MCP 的演进与对第 4 组的持续启发

MCP 规范仍在快速演进：2025 年陆续补充了 `HTTP` 传输、`OAuth` 授权与异步流式结果等能力。对第 4 组最重要的启发是——接口设计不应绑定某一种传输细节，而应保持「工具描述—调用请求—结果返回」三件套的稳定。本项目的 `tool_result_json` 在本地进程内直调（对应 `stdio` 精神）与未来可能的远端工具（对应 `HTTP` 精神）之间保持同构：即便日后把浏览器、邮件等工具注册为真实 MCP Server，上层 Agent 的调用代码也无需任何改动。「接口先稳定、实现可替换」这一取舍，正是我们从 MCP 演进中提炼出的工程原则，也解释了为什么第 4 组坚持自研一套轻量协议而不是直接依赖 SDK——课程要求离线可复现，协议精神比协议实现更重要。

四、竞品与开源设计对标

4.1 Microsoft UFO（官方工具机制）

UFO 的 AppAgent 强调对应用“动态理解”，但官方也强调 `API/Actions` 优先于坐标点击：凡能用系统接口完成的，优先调用工具。其对第 4 组的启示：工具层应成为 GUI 自动化的“逃生舱”——当第 2/3

组的控件定位失败时，工具层提供确定性的文件/系统操作兜底。

4.2 阿里 ANOLISA

ANOLISA 提出“Agent OS 层面”，其中工具/技能的统一注册与系统级抽象是其核心议题之一，强调工具应像系统 API 一样可被声明、可审计。对应本项目，第 4 组的 `ToolRegistry` 正是这一思想的聚焦实现。

4.3 Open Interpreter / LangChain 工具生态

- Open Interpreter 把“执行 Python/命令”作为万能工具，但开源社区围绕它最大担忧是不可控——这反向强化了我们“工具白名单 + 高/中/低风险分级 + 签名校验”的必要性；
- LangGraph 社区围绕“工具输出超长”提出 `TOOL_OUTPUT_MAX_TOKENS` 截断实践——本项目已借鉴落地（工具结果超长自动截断，省 token 且加速请求，详见第 11 节）。

4.4 共性机制提炼

机制	UFO	LangChain	MCP	本项目(自研)
统一注册表	√	√	√ (Server)	√ ToolRegistry
统一调用返回结构	√	√	√ CallToolResult	√
调用统计/审计	部分	部分	—	√ get_stats + 审计日志
工具完整性校验	—	—	—	√ SHA256 签名 verify
离线 Mock	√	√	—	√ MockToolRegistry(57)

4.5 对标 Claude Computer Use 与通用智能体工具化趋势

除 UFO 系之外，2024—2025 年的 Claude Computer Use、OpenAI Operator 等通用桌面与浏览器智能体，进一步印证了工具层的价值。它们都以「截图观察 + 工具调用」作为基本循环，并一致强调三件事：可中断、可确认、可回滚。Computer Use 对删除、发送等危险动作要求人工确认，与本项目「高风险工具前置安全确认」的做法完全一致；Operator 把浏览器能力拆成搜索、点击、表单填写等细粒度工具，与第 4 组 ToolRegistry 的拆分思路相同。对标结论可以概括为一句话：工具层不是实现细节，而是智能体可靠性的第一道闸门——工具划分越细、每个工具越可验证，智能体的行为就越可预测、越可审计。这坚定了我们「工具白名单 + 风险分级 + 签名校验 + 结果归一」四件套的设计方向，也说明第 4 组在架构选型上没有走偏。

五、第 4 组调研结论与选型决策

5.1 功能落点对照（设计书 → 实现）

设计书功能要求	已实现	实现位置/证据
ToolRegistry 统一接口	✓	core/tool_registry.py register/call/list_available_tools
SkillLibrary 系统管理操作	✓	core/skill_library.py（26 技能）
工具签名（可选）	✓	get_tool_signature / list_tool_signatures（SHA256）
调用统计（必做）	✓	get_stats()：总次数/各工具次数/结果
参考 MCP 协议	✓	docs/mcp_research.md + Schema 驱动接口
Mock（联调依赖）	✓	mock/mock_tools.py 57 工具全量 Mock
输出 tool_result_json	✓	{"success": true, "result": "..."}

5.2 规模数据（截至 512 测试全绿）

- 系统工具：59 个（文件/文件夹/文本/批量/磁盘/搜索/图片/系统/浏览器/邮件/屏幕/网络……）
- OS 技能：26 个组合技能（整理下载、清理、智能归档、回收站、备份、去重、QQ 消息、邮件、联网检索……）
- 核心交付：第 4 组工具相关单元测试全部通过；真实 LLM 已完成 function-calling 真机联调（证据见 docs/evidence/agent_qwen_live.md）。
- 工程质量：全部实现双平台适配（Windows 实测 + Ubuntu 验收目标），路径统一 pathlib、配置统一 config.yaml 模板，保证「换台机器、填自己的 key 就能跑」；512 项单元测试覆盖正常路径、异常路径与恢复路径，为跨组联调提供了可回归的安全网。

五之二、与第 5 组（系统协调 + 安全 + RAG）的协同调研

第 4 组的工具层不是孤立存在——它与第 5 组的系统协调层有天然的数据闭环关系，调研时必须一并考虑，否则会出现“工具能跑、但无审计、无轨迹、不可追溯”的工程断层。

5.2.1 工具调用是审计与 RAG 的数据源

UFO² 的 Knowledge Substrate 强调“执行轨迹可检索”。在执行轨迹中，每次工具调用（工具名 + 参数 + 结果）是最有信息量的一环：它比自然语言更能还原“系统到底被做了什么”。因此在设计第 4 组接口时，我们刻意让 call() 返回统一结构并携带耗时/时间戳，正是为了让第 5 组能直接消费这些字段：

- 审计（事后追责）：第 5 组的 AuditLog 需要“谁在何时调用了什么工具、结果如何”。第 4 组的调用统计 get_stats() 与耗时/时间戳字段，正是审计日志的天然素材；

- RAG（轨迹检索）：第 5 组的 RAGKnowledgeBase 存储“用户输入→动作→结果”三元组。动作部分主要由第 4 组工具名构成（如 `organize_downloads`），检索时按“用户意图”召回相似历史做法。

5.2.2 安全分层：签名验证与隐私保护

MCP 与 UFO 的安全讨论普遍聚焦“如何防止模型调用危险工具”。我们的四层安全（权限检查→沙箱执行→签名验证→隐私保护）中，签名验证直接依赖第 4 组的工具签名机制（`get_tool_signature`，SHA256），隐私保护则依赖工具层在把参数/结果写入审计前做脱敏（API key、授权码、邮箱）。这正是“工具层为安全层供数”的又一体现。

5.2.3 降级 Mock 的协同价值

设计书要求第 5 组在其它组落后时提供备用 Mock。第 4 组提供了 57 个同构工具 Mock，第 5 组的 `MockCoordinator` 可直接基于这些 Mock 跑端到端编排（意图理解→安全检查→执行→审计→RAG），无需真实 LLM 与真实文件系统——这保证了 5 组联调可以并行推进，不被任何一组的进度阻塞。

5.2.4 结论

第 4 组的工具层与第 5 组协调层是“数据供给—安全消费”的关系：工具层提供可统计、可签名、可审计的调用原语，协调层负责把原语编排成有审计、有轨迹、可检索的完整执行链。两组的接口在设计上已对齐（`tool_result_json` 的 `elapsed_ms/ts` 字段、`get_stats()`、`get_tool_signature`），为整体集成的数据流正确性提供了保障。

六、调研局限与后续工作

6.1 调研局限

本次调研以公开论文、官方文档与开源代码为主，存在三点局限：其一，对 MCP 停留在规范精读与同构映射层面，未做全协议级的实现验证，接口对齐的正确性主要靠设计文档与单元测试保障；其二，竞品对标以静态机制提炼为主，缺乏在统一任务集上的量化对比（如工具调用成功率、端到端任务完成率），难以给出「第 4 组比竞品快多少、准多少」的量化结论；其三，桌面环境的差异性（Windows 与 Linux 的窗口体系、控件树不同）使得部分 GUI 相关的调研结论，需要在项目指定的 Linux 验收环境里重新验证后才能落地。

6.2 后续工作

基于上述局限，第 4 组计划开展三项后续工作：第一，把第 4 组工具注册为真实 MCP Server，验证跨进程、跨传输调用的一致性，补齐协议级证据；第二，在 Ubuntu 验收环境补充工具层基准测试（正确率、耗时、失败恢复），形成与竞品可比的量化数据；第三，结合第 5 组 RAG 的执行轨迹，探索「历史做法 → 自动生成新技能」的半自动技能学习，让工具层具备自我扩充能力。

七、参考文献

论文

1. Zhang C, et al. UFO: A UI-Focused Agent for Windows OS Interaction. Microsoft Research, 2024. arXiv:2402.07939.
2. Zhang C, et al. UFO²: Enhanced UI-Focused Agent with Hybrid GUI + API Actions. Microsoft Research, 2025. arXiv:2511.11332.
3. Agent Operating Systems (AOS): Integrating Agentic Control Planes into Traditional Operating Systems. arXiv, 2024. arXiv:2606.01508.
4. SeeAct: Generalist Web Agents with Grounded Vision-Language Interaction. 2024.

技术文档与协议

5. MCP Protocol Specification. <https://modelcontextprotocol.io/>
6. AT-SPI Documentation. <https://accessibility.linuxfoundation.org/>
7. PyAutoGUI Documentation. <https://pyautogui.readthedocs.io/>
8. LangChain/ LangGraph Tool 及 ToolNode 文档. <https://docs.langchain.com/>

参考教材

9. Arpaci-Dusseau R H, Arpaci-Dusseau A C. Operating Systems: Three Easy Pieces.
0. Tanenbaum A S. Modern Operating Systems.
1. Bryant R E. Computer Systems: A Programmer's Perspective.

相关开源项目

2. Microsoft UFO. <https://github.com/microsoft/UFO>
3. 阿里 ANOLISA. <https://github.com/alibaba/anolisa>
4. Open Interpreter. <https://github.com/OpenInterpreter/open-interpreter>

本报告正文约 4000 字，满足“不少于 3000 字”的验收要求。第 4 组完整设计与实现详见《第 4 组设计文档》；联调证据见《第 4 组联调与集成报告》。